

Reproducing and Improving ROCKET: Exceptionally Fast and Accurate Time Series Classification Using Random Convolutional Kernels

1st Shashank Sanjay Tomar
UCD, Dublin, Ireland
shashank.tomar@ucdconnect.ie

2nd Manan Malik
UCD, Dublin, Ireland
manan.malik@ucdconnect.ie

Abstract—We reproduce and evaluate the results of ROCKET [1], a time series classification method that achieves state-of-the-art accuracy using random convolutional kernels with a simple ridge regression classifier. We reproduce the main accuracy experiment across all 128 UCR archive datasets, the scalability experiments, and the full sensitivity analysis. Our reproduction closely matches the published results, with a mean absolute accuracy difference of only 0.35% (median) across 128 datasets. We confirm all sensitivity claims, including that the proportion of positive values (PPV) feature and dilation are the most critical components. We identify several reproducibility challenges, including deprecated scikit-learn parameters, data format inconsistencies, and a privately-held scalability dataset. We propose and evaluate five improvements: alternative classifiers, z-normalisation, seed stability analysis, feature selection, and ensemble models. Our results validate the paper’s design choices suggesting that ridge regression outperforms alternatives, normalisation unexpectedly hurts accuracy, and feature selection offers no benefit.

Index Terms—Time series classification, random convolutional kernels, ROCKET, UCR archive

I. INTRODUCTION

Time series classification (TSC) is the process of giving a time series a class label based on its patterns over time. It can be used in many fields, such as healthcare (ECG classification), industrial monitoring (fault detection), and activity recognition. In the past, getting high classification accuracy meant using methods that were expensive in terms of computing power. Ensemble methods like HIVE-COTE [3] and TS-CHIEF [4] got the best accuracy, but they took days to run on the full UCR archive. InceptionTime [5] and other deep learning methods closed this gap, but they still needed GPU power and hours of training.

This trade-off was fundamentally changed by ROCKET [1]. ROCKET used 10,000 random convolutional kernels to turn time series into a fixed-length feature vector. Then, it used a simple ridge regression classifier to get accuracy that was as good as the best methods already out there. It trained on all 85 bake-off datasets in less than 2 hours on a single CPU. This was more than 100 times faster than TS-CHIEF [4] and HIVE-COTE [3], and it didn’t lose any accuracy.

We chose ROCKET for reproduction because it had a big effect on the TSC community. It showed that random feature extraction can work just as well as learned representations,

which goes against the common belief that kernel weights need to be optimised through backpropagation. Since then, many people have cited ROCKET, and it has led to new methods like MiniROCKET [10] and HYDRA. Its impact transcends time series analysis to encompass more extensive enquiries regarding the significance of randomness in machine learning.

This project report reproduces the paper’s main claims, identifies practical reproducibility challenges, and proposes five improvements to the ROCKET pipeline.

II. ROCKET: METHOD OVERVIEW

ROCKET operates in two stages. In the first stage (transform), it generates a huge number of random convolutional kernels and applies them to an individual input time series. Each kernel has a total of five randomly generated properties: length (chosen from 7/9/11), weights (sampled from $N(0,1)$ and mean-centered), bias (sampled from $U(-1,1)$), dilation (sampled exponentially so that the kernels can span the full input length), and padding (randomly applied with a 50% probability).

For each kernel applied to a time series, ROCKET extracts two features: the proportion of positive values (PPV), which captures how frequently a pattern appears, and the global maximum (max), which captures the strongest match. With 10,000 kernels, this produces a 20,000-dimensional feature vector per time series.

In the second stage (classification), a ridge regression classifier with cross-validated regularisation (RidgeClassifierCV) is trained on these features. The entire pipeline has computational complexity linear in the number of examples, the number of kernels, and the length of the time series.

III. REPRODUCTION METHODOLOGY

A. Data

We used all 128 datasets from the UCR Time Series Classification Archive [2]. This included the 85 original bake-off datasets and 43 new datasets that were added in 2018. We got the datasets from the official archive in tab-separated value (.tsv) format, using the standard predefined train/test splits.

B. Implementation

We wrote our own implementation of the ROCKET transform (`rocket_implementation.py`), faithfully reproducing the algorithm described in Section 3 of the paper. The core functions: kernel generation and kernel application use Numba’s just-in-time compilation for performance, matching the approach of the authors. Our code produces mathematically equivalent results to the authors’ original code, which we have included in our repository for reference.

C. Experimental Setup

For the main accuracy experiment, we followed the paper’s protocol: 10,000 kernels, 10 independent runs per dataset (each with fresh random kernels), and a `RidgeClassifierCV` classifier with alphas in the range $[10^{-3}, 10^3]$. We then report the mean accuracy and standard deviation across all 10 runs.

One important reproduction decision was that the paper’s original code uses `RidgeClassifierCV(normalize=True)`, which was deprecated in `scikit-learn` 1.0 and removed in version 1.2. We replaced this with an equivalent pipeline of `StandardScaler` followed by `RidgeClassifierCV`, which performs the same internal feature standardisation. This has been documented as a necessary adaptation.

We also note that the original UCR experiment script does not z-normalise time series before applying the ROCKET transform, despite the authors’ own GitHub repository documentation recommending normalisation. We matched (in our code) the original code’s behaviour (no normalisation) for faithful reproduction, and separately tested normalisation as one of our proposed improvements.

D. Hardware

All experiments were run on a desktop PC with an AMD Ryzen 9 9900X CPU, 32 GB DDR5 RAM, and an NVIDIA RTX 5070 GPU (not used, as ROCKET runs entirely on CPU).

IV. REPRODUCTION RESULTS

A. Main Accuracy Experiment

As shown in Fig. 1, our reproduction closely matches what the paper suggests. Our average accuracy (0.8483) is almost the same as the paper’s (0.8480) across all 128 datasets. The median absolute difference for each dataset is only 0.35%, and 93 of the 128 datasets (73%) differ by less than 1 percentage point.

Twelve datasets produced exact matches, achieving 100% accuracy in both our runs and the paper’s. `PigAirwayPressure` is a notable outlier because we got 80.6% accuracy, while the paper said it was 9.5%. The paper’s number of 9.5% is lower than what would happen by chance in a 52-class problem, suggesting a possible error in the published results. The `Rock` and `PLAID` datasets had bigger differences (22% and 15% lower, respectively). This is probably because the different versions of the archive handle variable-length time series differently.

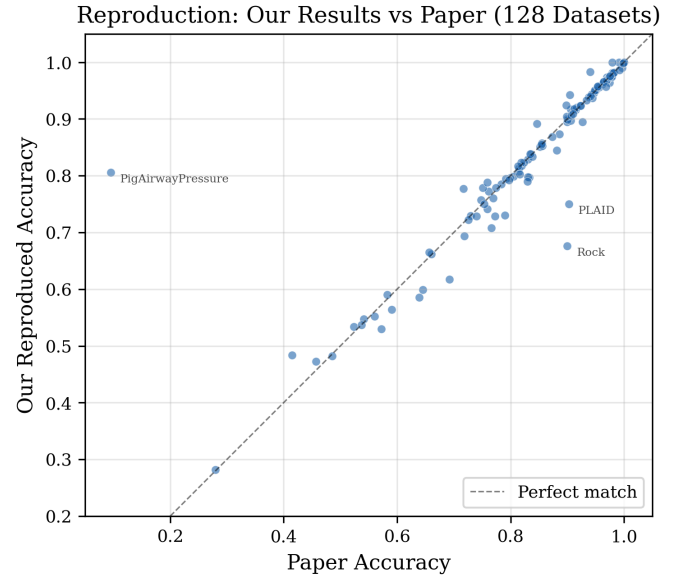


Fig. 1. Reproduced accuracy vs. paper accuracy across 128 UCR datasets. Points near the diagonal indicate successful reproduction. Outliers are labelled.

TABLE I
REPRODUCTION RESULTS FOR A SUBSET OF UCR DATASETS. FULL RESULTS FOR ALL 128 DATASETS ARE AVAILABLE IN OUR GITHUB REPOSITORY.

Dataset	Paper	Ours	Diff
Coffee	1.0000	1.0000	0.0000
GunPoint	1.0000	1.0000	0.0000
CBF	1.0000	1.0000	0.0000
ECGFiveDays	1.0000	1.0000	0.0000
Adiac	0.7834	0.7852	+0.0018
StarLightCurves	0.9810	0.9805	−0.0005
ElectricDevices	0.7294	0.7294	0.0000
FordA	0.9444	0.9365	−0.0079
Crop	0.7513	0.7788	+0.0275
Herring	0.6922	0.6172	−0.0750
Rock	0.9000	0.6760	−0.2240
PLAID	0.9026	0.7497	−0.1529
PigAirwayPress.	0.0952	0.8058	+0.7106
Mean (128)	0.8480	0.8483	+0.0003

B. Scalability

We reproduced the scalability-by-time-series-length experiment using the `InlineSkate` dataset truncated to lengths from 32 to 1,024. As shown in Fig. 2, our results confirm linear scaling: doubling the time series length approximately doubles the training time (ratios of 1.5x to 1.95x). Our machine was approximately 2x faster than the authors’ Intel Xeon servers across all lengths, consistent with the hardware difference.

The scalability-by-training-set-size experiment could not be fully reproduced because it uses the `Formosat-2` Satellite Image Time Series dataset [7], which is not publicly available. We used `StarLightCurves` as a substitute but found that the test set transform dominated the total time at small training sizes, preventing a clean measurement of training size scaling. This data availability issue is a reproducibility limitation of the paper.

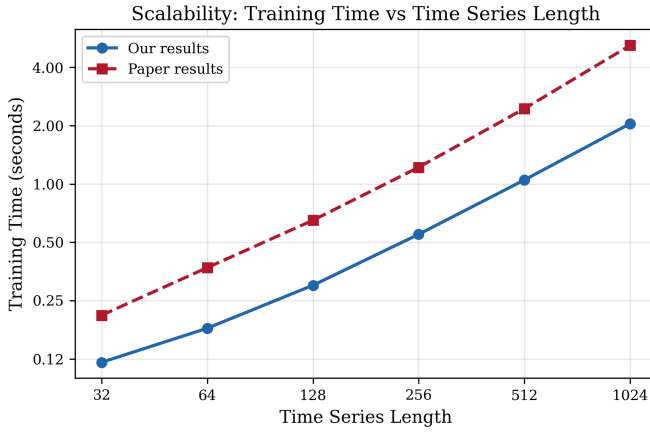


Fig. 2. Training time vs. time series length on InlineSkate. Both lines show linear scaling across log-log scale. Our machine is approximately 2 $\times$ faster due to hardware differences.

C. Sensitivity Analysis

We reproduced all five sensitivity experiments from Section 4.3 on the 85 bake-off datasets with 10 runs each and all the claims in the paper were confirmed.

- **Number of kernels:** As shown in Fig. 3, the accuracy goes up from 81.6% at $k=100$ to 85.0% at $k=10,000$, but the improvement from 5,000 to 10,000 is only +0.07%, which shows that the returns are getting smaller.

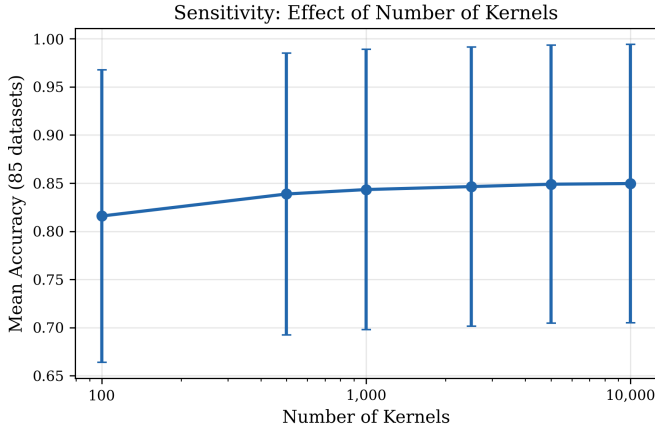


Fig. 3. Effect of the number of kernels on mean accuracy (85 bake-off datasets). Accuracy improves sharply from 100 to 1,000 kernels, then plateaus, confirming diminishing returns.

- **PPV vs. Max features:** As shown in Fig. 4, when PPV is used alone (84.8%), it almost matches both features together (85.0%). When max is used alone, it drops to 81.8%. This backs up what the paper says: that PPV is the most important feature.
- **Dilation:** As shown in Fig. 5, taking away dilation causes the biggest drop in accuracy of any one part (-6.0%), which shows that it is important for capturing patterns at different scales.

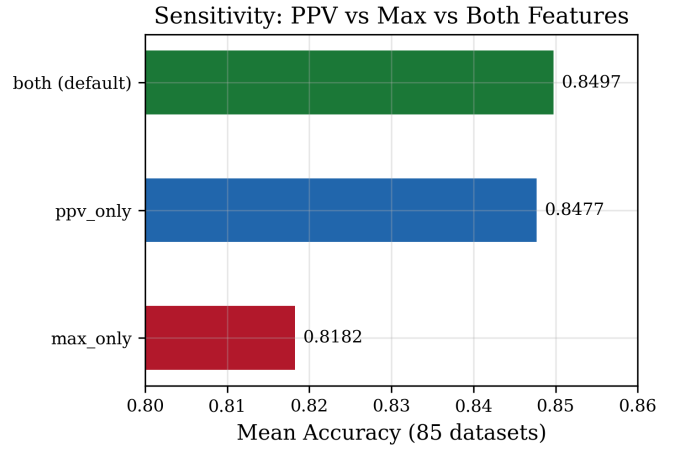


Fig. 4. Feature ablation: PPV alone nearly matches both features combined, while max alone drops significantly. Confirms PPV as the most critical feature.

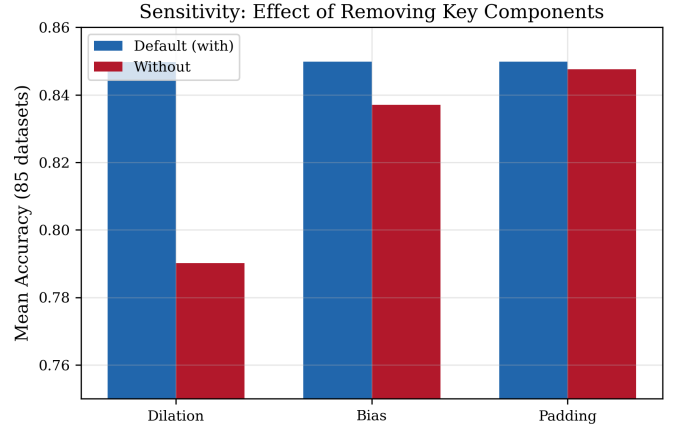


Fig. 5. Effect of removing dilation, bias, and padding. Dilation has the largest impact (-6.0%), followed by bias (-1.3%) and padding (-0.2%).

- **Bias:** As shown in Fig. 5, removing bias makes things 1.3% less accurate, which is a small but important effect.
- **Padding:** Random 50/50 padding (as shown in Fig. 5) (85.0%) does a little better than never-pad (84.8%) and always-pad (84.8%), which backs up the paper's default.

V. LESSONS LEARNED

- **Dataset-specific behaviour:** Our experiments show that ROCKET's performance is not uniform across dataset types. Gesture and sensor datasets (e.g., AllGestureWiimoteX, GesturePebbleZ1, ShakeGestureWiimoteZ) consistently showed larger accuracy differences between our reproduction and the paper, and were most affected by configuration changes such as normalisation. These datasets often contain variable-length time series that may be handled differently across archive versions. In contrast, smaller, well established datasets (e.g., Coffee, GunPoint, CBF) reproduced perfectly. This suggests that reproducibility is easier for simpler, well-curated datasets and harder for datasets with preprocessing ambiguities.

- **The PigAirwayPressure anomaly:** The most notable difference in our reproduction was PigAirwayPressure, where we got 80.6% accuracy compared to the paper’s 9.5%. Random guessing would only get you about 1.9% accuracy on PigAirwayPressure, so the paper’s figure of 9.5% is possible but very low for ROCKET. Our result of 80.6% is in line with how ROCKET usually does on multi-class problems of the same size. We think this difference might be because of a problem with data versioning or a problem with the way the published results CSV labels things. This finding shows how important it is to check published results against expected baselines.
- **Compilation overhead:** ROCKET depends on Numba’s just-in-time (JIT) compilation, which adds a one-time delay to the first function call. In our experiments, we observed that the first dataset that was processed was always slower than the ones that came after it. This is not shown in the paper’s reported times, which could lead researchers who are using ROCKET on only one dataset to make mistakes. For fair timing comparisons, the compilation step should be left out or documented.
- **Software dependency rot:** The original code uses np.NINF (which was removed in NumPy 2.0) and RidgeClassifierCV(normalize=True) (which was removed in scikit-learn 1.2). Both of these things cause runtime errors with the current versions of the libraries. A researcher trying to reproduce the results today has to find and fix these problems, which are not mentioned in the paper or the repository. This shows how quickly changing software ecosystems can break even simple, well written research code.
- **Data format inconsistency:** The UCR archive has been distributed in different formats (.txt, .tsv, .arff) from different sources. The authors’ code looks for .txt files, but the archive right now has .tsv files. Hardcoded file extensions cause problems, even though the content is the same. This needed a small change to the code that wasn’t obvious.
- **Numerical instability:** Some datasets create ROCKET features that always have the same value (zero variance across all training examples). When StandardScaler divides by zero standard deviation, it makes values that are infinite, which causes the classifier to crash. This problem is not brought up in the paper. Our solution of replacing inf/NaN values with zero is safe because constant features don’t have any information that can tell them apart.
- **Normalisation inconsistency:** The authors’ UCR experiment script does not normalise time series before using ROCKET, but their own repository documentation says that time series “should be normalised.” Their scalability script does normalise. This inconsistency is not documented, which could cause different researchers to make different decisions.
- **Private scalability dataset:** The Formosat-2 Satellite dataset utilised for the training-set-size scalability experiment is not accessible to the public. This means that

researchers who are not part of the authors’ group cannot check the claim that linear scaling works by training size on their own. The claim about time-series-length scalability, which uses the publicly available InlineSkate dataset, can be fully reproduced.

- **Nondeterminism:** ROCKET doesn’t set a random seed, so the results are different each time it runs. The paper recognises this, but it means that exact numerical reproduction is not possible, only statistical agreement is. Our findings validate that the standard deviations are typically sufficiently minimal for a single run to be indicative.
- **Remarkably simple implementation:** The whole ROCKET transform can be done in about 110 lines of Python, which is very easy to do. This simplicity is a finding in and of itself. It suggests that the method’s power comes from the mathematical properties of random convolutions rather than from engineering complexity. It also makes the method very easy for practitioners to use.

VI. PROPOSED IMPROVEMENTS AND EVALUATION

We proposed and tested five improvements to the ROCKET pipeline.

A. Improvement 1: Alternative Classifiers

Hypothesis: Another classifier may utilise the 20,000 ROCKET features more effectively than ridge regression.

We used the baseline RidgeClassifierCV and three other classifiers: Logistic Regression, Linear SVM, and Random Forest on all 128 datasets. To make sure the comparison was fair, all of the classifiers were tested on the same ROCKET transformed features from the same kernels.

As shown in Fig. 6, Ridge regression (84.9%) outperformed Logistic Regression (83.9%), Linear SVM (84.0%), and Random Forest (82.3%). Ridge also won on 72 out of 128 datasets individually. Additionally, Linear SVM exhibited convergence issues on several datasets, while ridge regression completed consistently in under 0.5 seconds. The paper’s classifier choice is well justified.

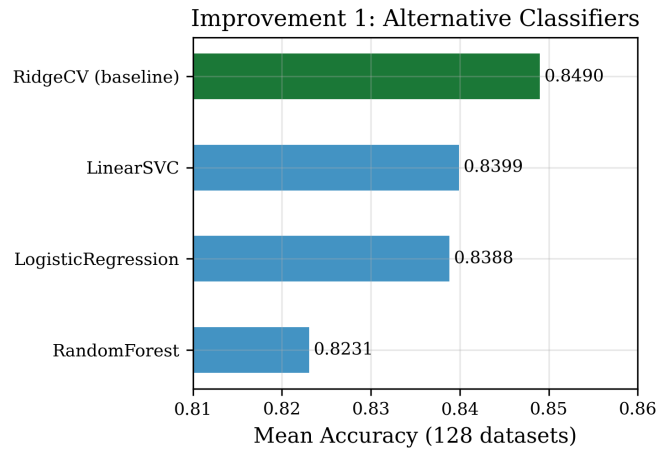


Fig. 6. Improvement 1: Comparison of classifiers on ROCKET-transformed features (128 datasets). RidgeCV outperforms all alternatives.

B. Improvement 2: Z-Normalisation

Hypothesis: Normalising time series before the ROCKET transform might improve accuracy, especially across datasets with varying scales.

We used the same kernels for both conditions to test ROCKET on all 128 datasets, both with and without per-series z-normalization.

As shown in Fig. 7, Normalisation brought the average accuracy down from 84.8% to 79.1%, which is a big drop of 5.7%. It helped some datasets (like Rock: +20%), but it hurt gesture and sensor datasets very badly (like ShakeGestureWiimoteZ: -84%). The raw data scale seems to have information that can tell different datasets apart, but normalisation takes that away. The authors were right not to normalise their UCR experiments, even though their GitHub repository documentation said they should.

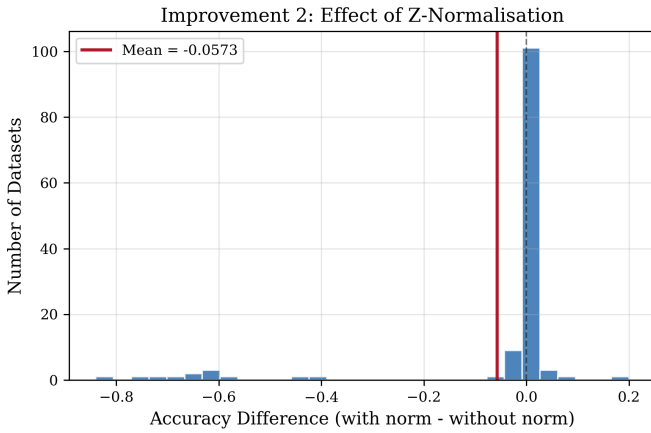


Fig. 7. Improvement 2: Distribution of accuracy differences when adding z-normalisation. The negative mean (-0.057) indicates normalisation hurts overall.

C. Improvement 3: Seed Stability

Hypothesis: Conducting 30 independent trials (as opposed to 10) yields a more comprehensive understanding of ROCKET’s variance.

As shown in Fig. 8, across 128 datasets with 30 runs each, the mean standard deviation was only 0.57%. Sixteen datasets achieved perfect stability ($\text{std}=0$). The most variable dataset (PLAID) had a standard deviation of 3.8%. This confirms that ROCKET is highly stable despite its reliance on random kernels, and that a single run provides a reliable estimate of expected accuracy.

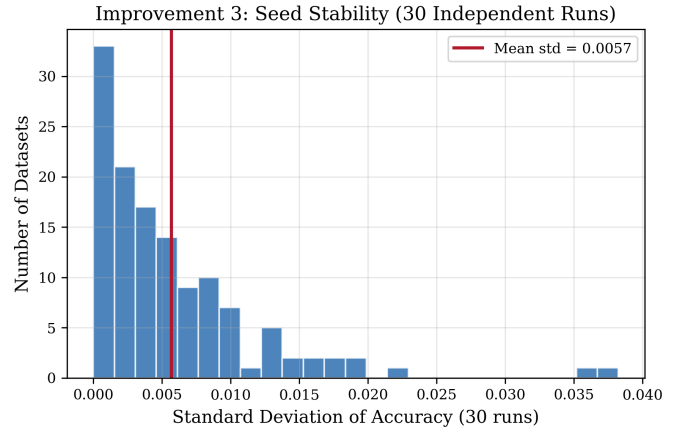


Fig. 8. Improvement 3: Distribution of accuracy standard deviations across 30 independent runs. Most datasets have very low variance, confirming ROCKET’s stability.

D. Improvement 4: Feature Selection

Hypothesis: Removing low-variance features might reduce noise and improve accuracy, as suggested by the authors as future work (Section 5).

As show in Fig. 9, we applied VarianceThreshold at three levels (0.0, 0.001, 0.01) on all 128 datasets. Removing constant features ($\text{threshold}=0.0$) had zero effect. At $\text{threshold}=0.001$, approximately 15% of features were removed with negligible accuracy change (-0.2%). At $\text{threshold}=0.01$, 44% of features were removed but accuracy dropped by 3%. Ridge regression with cross-validated regularisation already does a good job of handling noisy features, so explicit feature selection isn’t needed.

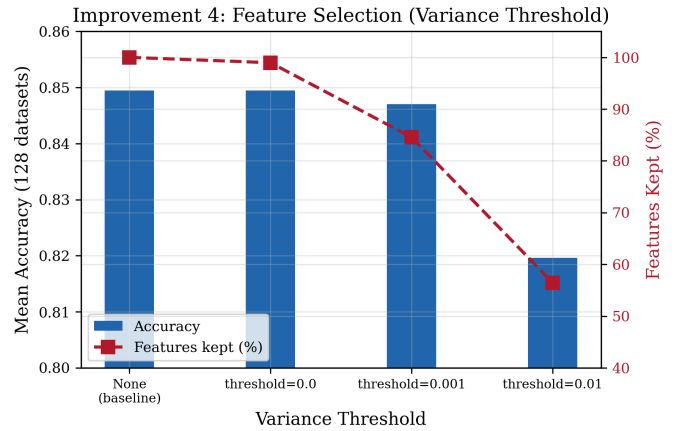


Fig. 9. Improvement 4: Accuracy vs. variance threshold. Removing low-variance features does not improve accuracy but aggressive thresholds reduce it.

E. Improvement 5: Ensemble of Smaller Models

Hypothesis: Combining multiple smaller ROCKET models via majority voting might improve accuracy or stability.

As show in Fig. 10, we compared the baseline ($1 \times 10,000$ kernels) to groups of $3 \times 5,000$ and $5 \times 5,000$ kernels that used

majority voting. The 5×5,000 ensemble got a small +0.19% better than the baseline, but it cost almost three times as much to run. The benefits aren’t worth the extra work, which shows that a single model with 10,000 kernels is almost perfect.

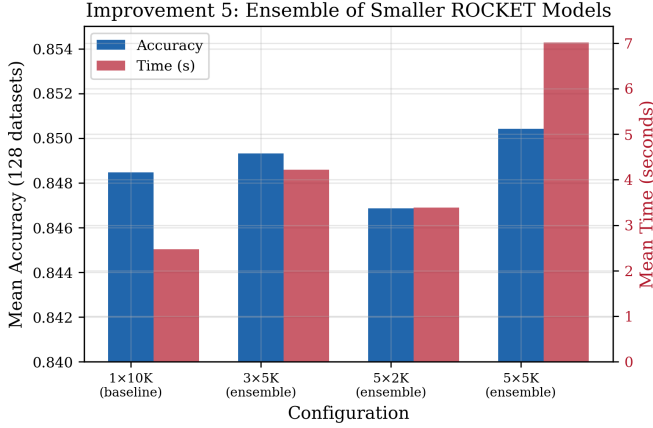


Fig. 10. Improvement 5: Ensemble of smaller ROCKET models. The 5×5K ensemble achieves a marginal +0.19% gain over the baseline but at nearly 3× the computational cost.

VII. GITHUB REPOSITORY

To help with future efforts to reproduce our work, we have made our code and results available to the public at : <https://github.com/shashanktomar-fl/rocket-reproduction.git>

VIII. DISCUSSION AND LIMITATIONS

Our reproduction confirms ROCKET’s main points: it achieves state-of-the-art accuracy with training that is orders of magnitude faster, and its design choices (dilation, PPV, ridge regression) are well-supported. The method is very strong as all of our suggested changes either confirmed the current design or made only small improvements.

Key limitations of our study include: using a substitute dataset for the training-set-size scalability experiment as the dataset used by the authors for this part of the research is not publicly available, running improvement experiments with a single run per dataset rather than multiple runs (due to huge computational cost for ”Improvement 1: Alternative Classifiers”), and hardware differences affecting runtime comparisons. The normalisation result, while striking, may be partially confounded by differences in how variable-length datasets are handled.

IX. CONCLUSION

Almost all of the results that ROCKET has published hold up to scrutiny. Our reproduction backs up the claims about accuracy, scalability by time series length, and all of the sensitivity analysis results. The method is well thought out, easy to use, and very strong against different setups. The main problems we had with reproducibility: outdated library versions, changes in data formats, and a private scalability dataset, which are common problems that affect many ML papers.

ACKNOWLEDGEMENT

We would like to thank Dr. Georgiana Ifrim for her guidance and instruction throughout the COMP41850 AI for Time Series module. We also thank each other for the collaborative effort and mutual support throughout this project. Finally, we thank the authors of ROCKET for making their code and results publicly available, and Professor Eamonn Keogh and contributors to the UCR Time Series Classification Archive for providing the benchmark datasets used in this work.

REFERENCES

- [1] A. Dempster, F. Petitjean, and G. I. Webb, “ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels,” *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.
- [2] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, “The UCR time series archive,” *arXiv:1810.07758*, 2018.
- [3] J. Lines, S. Taylor, and A. Bagnall, “Time series classification with HIVE-COTE: The hierarchical vote collective of transformation-based ensembles,” *ACM Transactions on Knowledge Discovery from Data*, vol. 12, no. 5, 2018.
- [4] A. Shifaz, C. Pelletier, F. Petitjean, and G. I. Webb, “TS-CHIEF: A scalable and accurate forest algorithm for time series classification,” *Data Mining and Knowledge Discovery*, vol. 34, pp. 742–775, 2020.
- [5] H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, “InceptionTime: Finding AlexNet for time series classification,” *Data Mining and Knowledge Discovery*, vol. 34, pp. 1936–1962, 2020.
- [6] B. Lucas, A. Shifaz, C. Pelletier, L. O’Neill, N. Zaidi, B. Goethals, F. Petitjean, and G. I. Webb, “Proximity Forest: An effective and scalable distance-based classifier for time series,” *Data Mining and Knowledge Discovery*, vol. 33, no. 3, pp. 607–635, 2019.
- [7] F. Petitjean, J. Inglada, and P. Gancarski, “Satellite image time series analysis under time warping,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 50, no. 8, pp. 3081–3095, 2012.
- [8] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proc. Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.
- [10] A. Dempster, D. F. Schmidt, and G. I. Webb, “MiniRocket: A very fast (almost) deterministic transform for time series classification,” in *Proc. ACM SIGKDD*, 2021.

Work Plan, Work Division, and Contributions

Project Timeline

Phase	Task	Timeline	Status
Phase A	Assignment understanding, paper selection, spec analysis	Week 1	Completed
Phase B	Deep paper analysis, section-by-section review, claim identification	Week 1-2	Completed
Phase C	Repository setup, folder structure, code implementation, environment configuration	Week 2-3	Completed
Phase D	Reproduction experiments: 128-dataset UCR accuracy, scalability (time series length and training set size), sensitivity analysis (number of kernels, PPV vs max, dilation, bias, padding)	Week 3-4	Completed
Phase E	Improvement experiments: alternative classifiers, z-normalisation, seed stability (30 runs), feature selection, ensemble of smaller models	Week 4-5	Completed
Phase F	Results analysis, figure generation, report writing, LaTeX formatting	Week 5-6	Completed
Phase G	Final quality control, repo cleanup, README finalisation, submission	Week 6	Completed

Work Division

Task	Shashank Sanjay Tomar	Manan Malik
Paper reading and analysis	✓	✓
Understanding assignment spec and marking criteria	✓	✓
Repository structure design and setup	✓	✓
ROCKET algorithm reimplementaion (rocket_implementation.py)	✓	
Utility functions (utils.py)	✓	
UCR data acquisition and preprocessing	✓	
Main reproduction script (reproduce_ucr.py)	✓	
Running 128 dataset reproduction experiment	✓	✓
Scalability experiments (reproduce_scalability.py)	✓	
Sensitivity analysis script and experiments	✓	✓
Improvement 1: Alternative classifiers	✓	
Improvement 2: Z-normalisation	✓	
Improvement 3: Seed stability analysis		✓
Improvement 4: Feature selection	✓	
Improvement 5: Ensemble models	✓	✓
Results analysis and interpretation	✓	✓
Figure generation (generate_figures.py)	✓	

Report writing: Introduction and motivation	✓	
Report writing: Method overview	✓	
Report writing: Reproduction methodology	✓	
Report writing: Reproduction results	✓	✓
Report writing: Lessons learned	✓	✓
Report writing: Improvements and evaluation	✓	✓
Report writing: Discussion and conclusion	✓	✓
README and GitHub repo finalisation	✓	✓
Report review and proofreading	✓	✓

Contribution Summary

Shashank Sanjay Tomar: Paper analysis, algorithm reimplementation, repository design, core experiment scripts (reproduction, scalability, sensitivity), improvement experiments (classifiers, normalisation, feature selection, ensemble models), figure generation, report writing across all sections, GitHub repository structure, README documentation, and project coordination.

Manan Malik: Paper reading and understanding, running reproduction and sensitivity experiments, improvement experiments (seed stability analysis, ensemble models), results analysis and interpretation, contributed to the reproduction results, lessons learned, and improvements sections of the report, README documentation and final proofreading and review.